Федеральное государственное автономное
образовательное учреждение высшего образования
«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

ОТЧЁТ

по лабораторной работе № 2
по дисциплине «Проектирование систем на кристалле»
«Настройка SCR1 и обработка исключений»
Вариант 9

Выполнила:
Пронина Анастасия Ильинична
Группа: P4130

Санкт-Петербург
2026

1 Цель и параметры задания

Изучить настройку архитектуры системы команд SCR1, механизм исключений и согласование аппаратных адресов входа с размещением программы компоновщиком.

Параметр	Вариант 9
Исключение	Illegal instruction — недопустимая инструкция
Тест	`isa/rv32mi/illegal.S`
Reset Vector	`0xF00`
Trap Vector	`0xB00`
Сообщение обработчика	`incorrect instruction`
Рабочая ветка	`lab_scr1_sim`

2 Окружение и подготовка репозитория

Использовано Docker-окружение на базе Ubuntu 24.04 с RISC-V GCC и Verilator 5.030. Симулятор собран из исходников согласно Dockerfile. Симуляция выполнялась в контейнере, исходники доступны в `/work/scr1`. Для ядра выбраны `CFG=MAX`, интерфейс `BUS=AHB`, архитектура RV32IMC.

Работа выполнена в ветке `lab_scr1_sim`.

3 Исходная проверка и выбор одного теста

До изменения обработчика и адресов сохранён базовый прогон ISA-набора:

```
make TARGETS="riscv_isa"
```

Все 56 тестов набора `riscv_isa` прошли успешно. Результаты сохранены в `report/results/baseline/test_results.txt`, журнал запуска — в `baseline.log` в том же каталоге.

Затем файл `sim/tests/riscv_isa/rv32_tests.inc` заменён строкой:

```
rv32_isa_tests := isa/rv32mi/illegal.S
```

Удалены остальные добавления списка, включая условные блоки расширений M и C. Параметр `TARGETS=riscv_isa` выбирает набор, а этот файл ограничивает его единственной программой. Отключение тестов M/C не отключает расширения ядра. Одиночный прогон до изменения обработчика также завершился `1/1 PASS`; его журнал сохранён в `report/results/illegal_before.log`.

4 Изменения аппаратных параметров и размещения программы

4.1 Адреса в RTL

В src/includes/scr1_arch_description.svh установлены:

```
SCR1_ARCH_RST_VECTOR = 'hF00;
SCR1_ARCH_MTVEC_BASE = 'hB00;
```

Первый параметр определяет адрес начала исполнения после сброса. Второй задаёт базу mtvec после сброса и фиксированные биты, если они предусмотрены конфигурацией. Он не размещает машинный код в памяти: размещение требуется согласовать отдельно.

4.2 Отдельные секции для обработчика и старта

В sim/tests/common/riscv_macros.h, внутри RVTEST_CODE_BEGIN, код обработчика и стартовый код разнесены:

```
.section .text.trap, "ax", @progbits
trap_vector:
    # обработчик исключений

.section .text.reset, "ax", @progbits
.globl _start
_start:
    # инициализация перед запуском теста
```

Флаги ax обозначают размещаемую в памяти исполняемую секцию. Текст сообщения находится в .rodata; переключение к нему выполнено через .pushsection/.popsection.

В sim/tests/common/link.ld заданы выходные секции:

```
.trap 0xB00 : {
    PROVIDE(__TEXT_START__ = .);
    KEEP(*(.text.trap))
} >RAM

.reset 0xF00 : {
    KEEP(*(.text.reset))
    *(.text.init)
} >RAM
```

KEEP сохраняет секции даже при удалении неиспользуемого кода. Относительные смещения .org для размещения обработчика и старта убраны. Адреса теперь закрепляет

компоновщик непосредственно, поэтому изменение размера обработчика не сдвигает _start. Принудительное отключение relaxation для этого не требуется. Служебные точки выхода симулятора SIM_EXIT=0xF4 и SIM_STOP=0xF8 сохранены.

4.3 Подготовка mtvec программой

Стартовый код устанавливает адрес обработчика также программно:

```
la t0, trap_vector
csrw mtvec, t0
```

Таким образом, аппаратная исходная база и программная запись согласованы с адресом метки trap_vector=0xB00. Директива ENTRY(_start) в linker script задаёт точку входа ELF, но не заменяет аппаратный Reset Vector.

5 Модификация обработчика trap_vector

При исключении аппаратура записывает причину в mcause, адрес проблемной инструкции в mepc и переходит по mtvec. Для недопустимой инструкции код причины равен 2.

В начале trap_vector добавлена проверка и посимвольный вывод:

```
csrr a4, mcause
li a5, CAUSE_ILLEGAL_INSTRUCTION
bne a4, a5, .Lafter_illegal_print
la a4, .Lillegal_message
li t0, 0xf0000000
.Lprint_illegal:
    lbu a5, 0(a4)
    beqz a5, .Lafter_illegal_print
    sb a5, 0(t0)
    addi a4, a4, 1
    j .Lprint_illegal
.Lafter_illegal_print:
    csrr a4, mcause
```

В исходном заголовке каждая продолжающая строка макроса завершается символом обратной косой черты; здесь приведён раскрытый ассемблерный фрагмент.

```
.Lillegal_message:
    .asciz "incorrect instruction\n"
```

.asciz добавляет нулевой байт конца строки. lbu читает очередной символ, sb отправляет его в порт симулятора 0xF0000000. Этот адрес используется также в sc_print.c. Вызов C-функции и настройка стека для печати не требуются.

После вывода mcause читается повторно, поскольку a4 в цикле использовался как указатель. Код ecall не равен 2, поэтому сообщение при завершении теста повторно не печатается. Существующие проверки ecall, передача управления mtvec_handler и выход через sc_exit сохранены. Регистры a4, a5, t0 здесь являются рабочими; для выбранного теста его собственный обработчик заново задаёт используемые t0 и t1. Изменение не является универсальным обработчиком с сохранением полного контекста для произвольной программы.

6 Как продолжается тест после исключения

Файл dependencies/riscv-tests/isa/rv32mi/illegal.S подключает общий код из ../rv64mi/illegal.S, переопределяя макрос режима на RV32. Первая проверка выполняет:

```
li TESTNUM, 2
bad2:
    .word 0
    j fail
```

Нулевое машинное слово должно вызвать исключение. Переход j fail поймал бы ошибку, если бы процессор продолжил обычное исполнение.

Общий trap_vector после сообщения передаёт управление mtvec_handler, определённому самим тестом. Объявление .weak mtvec_handler лишь делает символ необязательным; его фактический адрес определяется меткой в программе. Обработчик теста проверяет mcause, mepc и допустимое содержимое mtval, после чего для bad2 выполняет:

```
addi t0, t0, 8
csrw mepc, t0
mret
```

Так пропускаются четыре байта .word 0 и четыре байта j fail. В тесте включена .option norvc, поэтому последующая команда перехода имеет длину четыре байта.

SCR1 поддерживает только Machine mode. Проверка наличия Supervisor mode не подтверждает его поддержку, поэтому оставшиеся Supervisor-проверки пропускаются переходом на pass. Макрос успеха устанавливает a0=0 и выполняет ecall. Это отдельное исключение с mcause=11; общий обработчик направляет его на _report, затем sc_exit. Операционной системы здесь нет.

7 Итоговая симуляция и проверка адресов

Для финальной проверки запущено из корня SCR1 внутри контейнера:

```
make run_verilator CFG=MAX BUS=AHB TARGETS=riscv_isa TRACE=1
```

После изменения конфигурации или для полностью чистой пересборки перед запуском используется `make clean`. Для повторного запуска отчёта использована отдельная конфигурационная папка `verilator_AHB_MAX...`, не затрагивающая ранее записанную диаграмму лабораторной 3.

Фактический вывод:

```
---Test: illegal.hex
incorrect instruction
Test passed
Summary: 1/1 tests passed
illegal.hex    OK    PASS
```

Проверялось содержимое `test_results.txt`, а не только код возврата `make`: используемые сборкой конвейеры с `tee` могут скрывать ошибку отдельной команды.

По `illegal.dump` подтверждены адреса:

Метка	Адрес	Назначение
`trap_vector`	`0xB00`	Общий обработчик
`_start`	`0xF00`	Старт после сброса
`_run_test`	`0x1000`	Тело теста
`bad2`	`0x1004`	Недопустимая инструкция
`mtvec_handler`	`0x1200`	Обработчик теста
`pass`	`0x110C`	Успешное завершение

После обработки `bad2` адрес возврата составляет `0x1004+8=0x100C`. Завершающий `ecall` находится по `0x1118`; его причина отличается от Illegal instruction.

8 Трассы MPRF и CSR

Штатный трассировщик этой версии SCR1 создаёт единый файл `tracelog_core_0.log`. Его реализация находится в `src/core/pipeline/scr1_tracelog.sv`. Из общего лога без изменения строк выделены отдельные материалы:

- `trace_mprf_diff_0.log`: строки записи целочисленных регистров, определяемые по колонке `Reg` с именами `xNN_alias`;
- `trace_csr_0.log`: строки снимков CSR, которые напечатаны штатным трассировщиком при исключениях и других предусмотренных им событиях.

Исходный объединённый лог сохранён рядом с выборками. Для выделения использована колонка `Reg`: имена целочисленных регистров `xNN_alias` для MPRF и имена CSR для второй выборки. Заголовки выборок указывают их происхождение; строки данных не изменены. Отдельная CSR-выборка не заявляется полным журналом всех записей CSR: её полнота ограничена событиями штатного трассировщика.

Подтверждённые снимки:

CSR	Illegal instruction	Завершающий ecall
mcause	0x00000002	0x0000000B
mepc	0x00001004	0x00001118
mtval	0x00000000	0x00000000
mstatus	0x00001800	0x00001800
Следующий PC в трассе	0x00000B00	0x00000B00

В режиме Verilator колонка Time исходной текстовой трассы содержит счётчик тактов time_cnt, а не физическое время в ps. Для определения проблемной инструкции использованы mepc и дизассемблирование; колонка Instr штатной трассы сама по себе не используется как единственное подтверждение причинного адреса.

9 Сохранённые файлы и соответствие заданию

В корневой папке results/ репозитория находятся:

- test_results.txt — итог одиночного теста;
- illegal.dump — дизассемблирование;
- trace_mprf_diff_0.log — записи MPRF;
- trace_csr_0.log — снимки CSR;
- tracelog_core_0.log — исходная объединённая трасса;
- sim_results.txt — журнал последнего запуска.

Подробный отчёт, LaTeX и PDF размещены в report/. Предусмотренный заданием файл lab_scr1_sim/README.md содержит описание работы и ссылки на эти материалы.

Исходная проверка документирована для ISA-набора. Если под формулировкой «все тесты» требуется запуск всех наборов репозитория, включая compliance и бенчмарки, сохранённые 56 результатов этого не подтверждают и такая проверка должна быть выполнена отдельно.

10 Сборка программы и загрузка в модель памяти

Сборка программного теста и сборка RTL — разные процессы. RISC-V GCC обрабатывает ассемблерный файл с расширением .S: препроцессор раскрывает включаемые заголовки и макросы, затем ассемблер создаёт объектный файл. Компоновщик объединяет его с необходимым кодом и применяет link.ld. Получается ELF с машинными инструкциями, данными, адресами символов и точкой входа.

Правила в sim/tests/riscv_isa/Makefile затем используют objcopy для получения illegal.hex и objdump -D -w -x -S для получения illegal.dump. Дизассемблирование позволяет проверить адреса до запуска симуляции. Линковщик сам не записывает программу в модель памяти.

В src/tb/scr1_memory_tb_ahb.sv загрузку выполняет:

```
if (test_file_init) $readmemh(test_file, memory);
```

Тестовое окружение читает HEX в массив модели памяти. После сброса ядро выбирает инструкции начиная с аппаратного Reset Vector. Значения `0xF00` в RTL и адреса `_start` в ELF должны совпадать; аналогичное согласование требуется для `0xB00` и обработчика. Заголовки ELF процессор не исполняет: интерфейс памяти передаёт ему машинные слова из уже загруженного образа.

11 Последовательность воспроизведения проверки

Команды ниже выполняются из корня SCR1 в подготовленном Linux-контейнере. Компилятор RISC-V и Verilator должны быть доступны через `PATH`.

```
verilator --version
make clean
make run_verilator CFG=MAX BUS=AHB TARGETS=riscv_isa TRACE=1
```

Перед сборкой должны быть подготовлены зависимости с исходниками тестов. Команда `make clean` удаляет предыдущую сборку, поэтому нужные журналы и дизассемблирование следует предварительно сохранять вне `build/`.

Для используемых параметров результаты создаются в каталоге:

```
build/verilator_AHB_MAX_imc_IPIC_1_TCM_1_VIRQ_1_TRACE_1/
```

Признаки корректного результата проверяются совместно:

1. В `test_results.txt` ровно один тест — `illegal.hex`, сборка `OK`, симуляция `PASS`.
2. В журнале напечатана строка `incorrect instruction` и итог `1/1 tests passed`.
3. В `illegal.dump` адреса `trap_vector` и `_start` равны `0xB00` и `0xF00`.
4. Трасса CSR содержит `mcause=2`, `mepc=0x1004` и переход к `0xB00`; последующий `ecall` имеет причину 11.

Для получения временной диаграммы используется режим генерации waveform:

```
make run_verilator_wf CFG=MAX BUS=AHB TARGETS=riscv_isa TRACE=1
```

Файл `simx.vcd` открывается в GTKWave. На предоставленном снимке видны тактовый сигнал `clk`, признак действительной команды `exu_queue_vd` и адрес `pc_curr_ff` блока EXU.

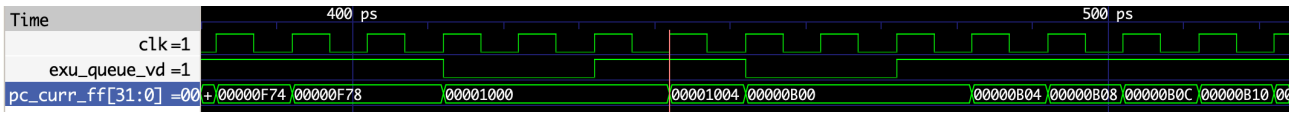


Рис. 1: Переход от недопустимой инструкции по адресу 0x1004 к общему обработчику по адресу 0xB00. Снимок GTKWave.

Последовательность значений PC на рисунке включает `0x1000`, `0x1004`, затем `0xB00`, `0xB04`, `0xB08` и последующие адреса обработчика. Адрес `0x1004` соответствует нулевому машинному слову в точке `bad2`; переход к `0xB00` согласуется с установленным Trap Vector. Причина исключения подтверждается отдельной трассой CSR: `mcause=2`, `mepc=0x1004`. Признак `exu_queue_vd` показывает наличие действительной команды, а не факт её завершения; значения PC при нулевом признаке не следует трактовать как исполнение инструкции. На этом снимке нет сигналов записи в порт `0xF0000000`, поэтому вывод строки подтверждается журналом симуляции, приведённым выше.

11.1 Вывод сообщения через порт симулятора

На снимке ниже шина `dmem_hwdata` отображена в ASCII-представлении: последовательно читаются символы `incorrect instruction`.

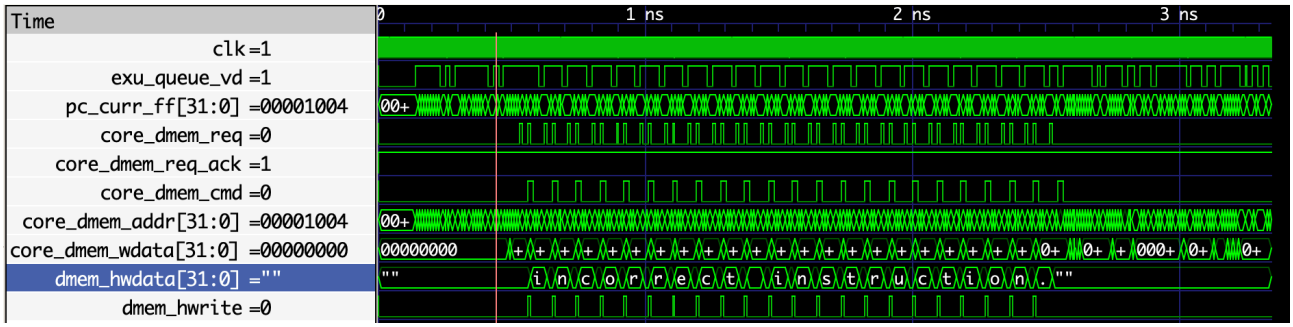


Рис. 2: Посимвольный вывод сообщения обработчика.

Обработчик читает байты строки и выполняет `sb` по адресу `0xF0000000`. Начало сообщения кодируется байтами `69 6E 63 6F 72 72 65 63 74 20` — `incorrect` и пробел. На стороне ядра запрос принимается на фронте `clk`, когда `core_dmem_req=1` и `core_dmem_req_ack=1`. `core_dmem_cmd` различает чтение символа и запись в порт.

Сигналы `dmem_hwrite` и `dmem_hwdata` относятся к AHB. Адресная фаза и фаза данных разделены, поэтому их нельзя автоматически сопоставлять с текущим запросом ядра в тот же момент. В данном масштабе адресная шина слишком плотная для чтения каждого адреса; адрес порта подтверждается листингом, а снимок показывает передаваемую строку. Значение шины вне действительной передачи не означает новую запись.

12 Вывод

Обработка Illegal instruction дополнена выводом заданной строки с сохранением логики проверочного теста. Аппаратные Reset Vector и Trap Vector согласованы с размещением секций программы: `_start=0xF00`, `trap_vector=0xB00`. По дизассемблированию и

снимкам CSR подтверждены правильный вход в обработчик, возврат после ожидаемого исключения и завершение через `ecall`. Финальная симуляция единственного теста завершилась PASS.